

THREAT INTELLIGENCE AGGREGATOR

*Design and Implementation of a Modular Threat Intelligence
Processing Toolkit*

Project Report

Prepared by

Divyanshi Kishore

Submitted To

Unified Mentor

Submission Date

July 2026

Acknowledgement

I would like to express my sincere gratitude to Unified Mentor for providing the opportunity to work on the Threat Intelligence Aggregator project. This project offered valuable practical experience in cybersecurity, software engineering, and threat intelligence processing.

Throughout the development of this project, I gained hands-on experience in designing a modular Python application capable of collecting, validating, normalizing, correlating, and exporting threat intelligence indicators from multiple data sources. The project strengthened my understanding of clean software architecture, modular design, automated testing, logging, and defensive cybersecurity workflows.

I also appreciate the availability of public threat intelligence feeds and open-source technologies that made it possible to develop and evaluate this toolkit using real-world threat data.

This project has significantly improved my technical skills in Python development, threat intelligence processing, software testing, version control with Git, and technical documentation. The knowledge and experience gained during this project will be valuable for future cybersecurity and software development work.

Abstract

Threat intelligence plays a critical role in modern cybersecurity by helping organizations identify, analyze, and respond to malicious infrastructure before attacks can cause significant damage. However, threat intelligence feeds are often distributed across multiple sources, represented in different formats, and contain duplicate or inconsistent Indicators of Compromise (IOCs), making manual analysis inefficient.

The objective of this project was to design and implement a modular Threat Intelligence Aggregator capable of collecting threat intelligence from multiple external sources, parsing different feed formats, validating and normalizing indicators, removing duplicate entries, correlating related indicators, and exporting the processed data into multiple security-focused formats.

The application was implemented in Python using a modular architecture based on clean design principles. The system supports multiple IOC types, including IPv4, IPv6, IPv4 CIDR, domains, URLs, MD5, SHA-1, and SHA-256 hashes. Threat intelligence feeds are processed through a structured pipeline consisting of feed collection, parsing, validation, normalization, deduplication, correlation, reporting, and export generation.

The project generates multiple output formats, including JSON datasets, CSV exports, correlation reports, threat intelligence summaries, and category-specific blocklists that can be integrated into defensive security tools.

Comprehensive automated testing was performed using pytest, with all 76 test cases passing successfully. Runtime verification confirmed successful processing of all configured feeds and generation of the expected output files.

The completed Threat Intelligence Aggregator demonstrates the practical application of software engineering principles and defensive cybersecurity techniques while providing a scalable foundation for future enhancements such as STIX/TAXII integration, database support, web-based dashboards, and external threat intelligence enrichment.

1. Introduction

Threat intelligence plays a significant role in modern cybersecurity by enabling organizations to identify malicious infrastructure, detect emerging threats, and strengthen defensive mechanisms. Every day, numerous security organizations publish threat intelligence feeds containing Indicators of Compromise (IOCs) such as IP addresses, domains, URLs, and file hashes. These indicators help security analysts identify known malicious activity and improve incident response.

However, publicly available threat intelligence feeds often differ in format, quality, and structure. Some feeds are distributed as CSV files, others as JSON documents, while many are available as plain text files. The same IOC may appear across multiple feeds, and some entries may be invalid, outdated, or duplicated. These inconsistencies make manual processing inefficient and increase the likelihood of errors. Security teams therefore require a reliable mechanism to automatically collect, validate, normalize, and organize threat intelligence into a consistent and usable format.

The Threat Intelligence Aggregator was developed to address these challenges by providing a modular Python-based toolkit that automates the complete threat intelligence processing pipeline. The system downloads threat feeds from multiple sources, parses different file formats, validates IOC formats, removes duplicate entries, correlates indicators from different feeds, and exports the processed intelligence into structured reports and defensive blocklists.

The project has been designed using modern software engineering practices to ensure maintainability, scalability, and reliability. The implementation follows Clean Architecture principles, SOLID design principles, dependency injection, modular component organization, comprehensive logging, type hints, and automated testing. This architectural approach enables individual components to be modified or extended with minimal impact on the overall system.

The completed implementation supports multiple IOC categories, including IPv4 addresses, IPv6 addresses, IPv4 CIDR ranges, domain names, URLs, MD5 hashes, SHA1 hashes, and SHA256 hashes. The processed intelligence can be exported in JSON and CSV formats, along with correlation reports, threat intelligence reports, and IOC-specific blocklists that can be used in defensive cybersecurity environments.

The project demonstrates how a structured, modular, and automated approach can improve the quality and usability of publicly available threat intelligence while providing a strong foundation for future enhancements such as STIX/TAXII integration, REST APIs, databases, dashboards, and CI/CD pipelines.

2. Problem Statement

Cyber Security analysts and security teams depend on threat intelligence feeds to identify malicious infrastructure, detect known threats, and strengthen organizational defenses. Numerous security organizations publish free and open-source threat intelligence feeds containing Indicators of Compromise (IOCs) such as IP addresses, domain names, URLs, and cryptographic file hashes. While these feeds provide valuable information, they often differ significantly in structure, format, and quality.

Threat intelligence data is commonly distributed as TXT, CSV, or JSON files, each following different schemas and naming conventions. As a result, collecting data from multiple sources requires separate parsing logic for each format. In addition, the same IOC frequently appears in multiple feeds, creating duplicate entries that increase data volume without adding useful information. Some feeds may also contain malformed or invalid indicators, reducing the reliability of the collected intelligence.

Manual collection and processing of multiple threat intelligence feeds is time-consuming, error-prone, and difficult to maintain. Without a standardized processing pipeline, security analysts must spend additional effort validating, cleaning, normalizing, and organizing the collected data before it can be used effectively. This process becomes increasingly challenging as the number of feeds grows.

The primary problem addressed by this project is the development of a modular and automated system capable of downloading, parsing, validating, normalizing, deduplicating, correlating, and exporting threat intelligence obtained from multiple external sources. The system aims to provide high-quality, structured outputs that can support defensive cyber security operations while maintaining a clean, maintainable, and extensible software architecture.

By automating these tasks, the Threat Intelligence Aggregator reduces manual effort, improves data consistency, eliminates redundant indicators, and generates organized outputs suitable for reporting, analysis, and defensive security applications.

3. Objectives

The primary objective of the Threat Intelligence Aggregator is to design and implement a modular, maintainable, and automated threat intelligence processing toolkit capable of collecting and processing Indicators of Compromise (IOCs) from multiple publicly available threat intelligence feeds. The project focuses on improving the efficiency, consistency, and usability of threat intelligence by automating the complete processing workflow while following modern software engineering practices.

The specific objectives of the project are as follows:

- To design a modular software architecture based on Clean Architecture and SOLID principles.
- To automate the downloading of threat intelligence feeds from multiple external sources.
- To support multiple input formats, including TXT, CSV, and JSON.
- To parse and extract Indicators of Compromise from different feed structures.
- To validate IOCs and eliminate invalid or malformed entries.
- To normalize IOC data into a standardized internal representation.
- To identify and remove duplicate indicators collected from different threat feeds.
- To correlate indicators based on their occurrence across multiple sources.
- To generate structured threat intelligence reports summarizing the processed data.
- To export processed intelligence in JSON and CSV formats.
- To generate IOC-specific blocklists for IPv4, IPv6, domains, URLs, MD5, SHA1, SHA256, and IPv4 CIDR indicators.
- To provide a command-line interface for executing the threat intelligence processing workflow.
- To maintain comprehensive logging for monitoring, debugging, and troubleshooting.
- To verify the correctness and reliability of the implementation through automated unit testing.
- To develop a scalable foundation that supports future enhancements without requiring major architectural changes.

These objectives collectively ensure that the system provides a reliable, extensible, and production-oriented solution for processing publicly available threat intelligence while maintaining high standards of software quality and maintainability.

4. Scope

The scope of the Threat Intelligence Aggregator is to provide an automated and modular solution for collecting, processing, validating, and exporting publicly available threat intelligence. The project focuses on building a reliable processing pipeline that converts raw threat intelligence feeds into structured and usable outputs suitable for defensive cybersecurity operations.

The implemented system automates the complete workflow from downloading threat intelligence feeds to generating reports and IOC-specific blocklists. The project emphasizes maintainability, extensibility, and software quality by adopting modern software engineering practices and a modular architecture.

Included Scope

The current implementation includes the following capabilities:

1. Automated downloading of threat intelligence feeds from multiple external sources.
2. Feed configuration and management using YAML configuration files.
3. Support for multiple feed formats, including TXT, CSV, and JSON.
4. Parsing of threat intelligence data using dedicated parser modules.
5. Validation of Indicators of Compromise (IOCs) before processing.
6. Normalization of IOC data into a consistent internal format.
7. Removal of duplicate indicators collected from multiple sources.
8. Correlation of IOCs across different threat intelligence feeds.
9. Generation of structured threat intelligence reports.
10. Export of processed intelligence in JSON and CSV formats.
11. Generation of IOC-specific blocklists for:
 - IPv4
 - IPv6
 - IPv4 CIDR
 - Domain

- URL
- MD5
- SHA1
- SHA256

12. Comprehensive application logging.

13. Command-line interface (CLI) for executing the processing pipeline.

14. Automated unit testing to verify functionality and reliability.

Excluded Scope

The following features are intentionally outside the scope of Version 1.0.0 and have been identified as potential future enhancements:

1. STIX/TAXII integration.
2. Email IOC parsing.
3. IOC severity or confidence scoring.
4. Database storage for persistent IOC management.
5. REST API for remote access.
6. Web-based dashboard for visualization and monitoring.
7. Docker containerization.
8. Continuous Integration and Continuous Deployment (CI/CD) pipeline.

By limiting the scope to the core threat intelligence processing workflow, the project delivers a stable, thoroughly tested, and maintainable solution while providing a strong architectural foundation for future expansion.

5. Technologies Used

The Threat Intelligence Aggregator has been developed using Python and several standard libraries to create a modular, maintainable, and efficient threat intelligence processing toolkit. The selected technologies support data collection, parsing, validation, reporting, testing, and overall software quality while keeping the project lightweight and platform-independent.

Technology	Purpose
Python 3	Primary programming language used for implementing the complete application.
PyYAML	Reads and parses YAML configuration files containing feed definitions.
pathlib	Simplifies cross-platform file and directory management
json	Parses JSON feeds and generates JSON output files .
typing	Adds type hints to improve code readability and maintainability
pytest	Performs automated unit testing and validation of application modules
Request	Downloads threat intelligence feeds over HTTP/HTTPS
argparse	Provides the command-line interface (CLI) for executing the application with different options.
csv	Reads CSV-based threat intelligence feeds and generates CSV exports .
logging	Records application events, execution status, warnings, and errors for monitoring and debugging.
Dataclasses	Defines structured data models for Indicators of Compromise (IOCs).
unittest.mock	Mocks external dependencies during testing to ensure isolated and reliable unit tests.

Development Tools

The following tools were used during development:

TOOL	PURPOSE
Visual Studio Code	Source code development and debugging.
Git	Version control and source code management.
GitHub	Repository hosting, version tracking, and project collaboration.
Command Prompt Terminal	Running the application, executing tests, and managing the project.

Programming Language

Python was selected because of its readability, extensive standard library, strong ecosystem, and widespread adoption in cybersecurity. Its modular programming capabilities and rich support for file handling, networking, and automation make it well suited for implementing a threat intelligence processing system.

Testing Framework

The project uses the **pytest** framework to verify the correctness of individual modules and overall system behavior. During final validation, all automated tests completed successfully with the following result:

Total Tests: 76

Passed: 76

Failed: 0

This testing process helped ensure the stability, correctness, and reliability of the implemented functionality before the final release.

6. Software & Hardware Requirements

The Threat Intelligence Aggregator is designed to be lightweight and platform-independent. The application does not require specialized hardware and can be executed on most modern desktop or laptop systems running Python. Since the project processes publicly available threat intelligence feeds over the internet, a stable network connection is recommended during execution.

Software Requirements

The following software components are required to develop and execute the application:

SOFTWARE	REQUIREMENT
Operating System	Windows 10/11, Linux, or macOS
Python	Python 3.x
Packet Manager	pip
Code Editor	Visual Studio Code (Recommended)
Testing Framework	pytest
Version Control	Git
Internet Connection	Required for downloading threat intelligence feeds

Python Dependencies

The project depends on a small number of external Python packages along with several built-in Python modules.

External Packages

- requests
- PyYAML
- pytest

Standard Library Modules

- argparse
- csv
- json
- pathlib

- logging
- typing
- dataclasses
- datetime
- collections
- unittest.mock

These standard library modules are included with Python and do not require separate installation.

Hardware Requirements

The application has minimal hardware requirements because it primarily performs file processing, validation, and report generation.

Component	Minimum Requirement
Processor	Dual-Core Processor
RAM	4 GB
Storage	500 MB Available Disk Space
Network	Internet Connection
Display	1366 × 768 Resolution or Higher

Execution Environment

The application is executed through the command-line interface (CLI). During project validation, the toolkit successfully processed three configured threat intelligence feeds and produced the following results:

Total Feeds Processed: 3

Successful Feeds: 3

Total IOCs Processed: 6,289

Average Runtime: Approximately 9 seconds

7. System Architecture

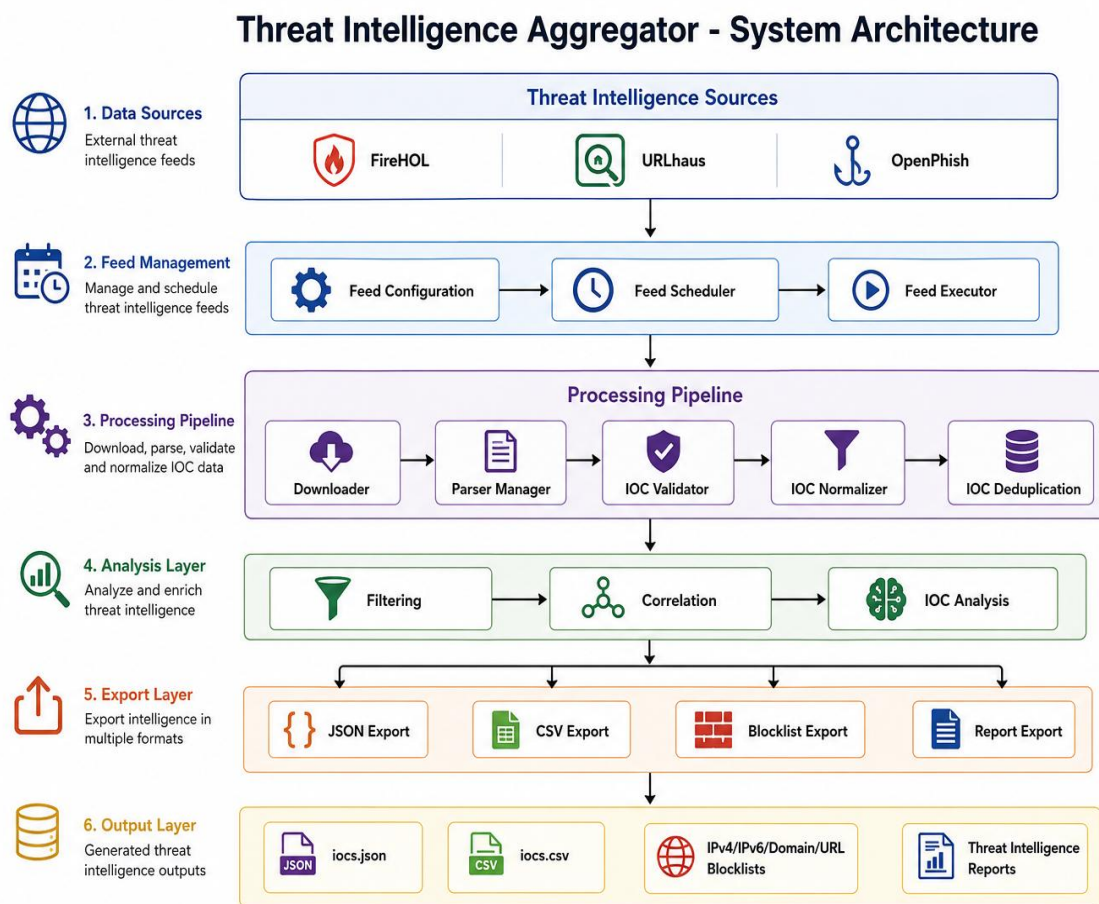


Figure 1. Overall System Architecture

The **Threat Intelligence Aggregator** follows a modular architecture based on Clean Architecture and SOLID principles. Each major functionality is implemented as an independent module with a well-defined responsibility. This approach improves maintainability, simplifies testing, and allows future enhancements without affecting unrelated components.

The application follows a sequential processing pipeline beginning with feed configuration and ending with the generation of reports and blocklists. Data flows through several processing stages where it is downloaded, parsed, validated, normalized, deduplicated, correlated, and exported.

The major architectural components are:

1. Configuration Management
2. Feed Manager
3. Downloader
4. Parser Manager
5. IOC Validator
6. IOC Normalizer
7. Deduplication Engine
8. Correlation Engine
9. Exporters
10. Reporting Module
11. Command-Line Interface (CLI)
12. Logging System

Each component performs a single responsibility, making the codebase easier to understand, maintain, and extend.

8. Workflow

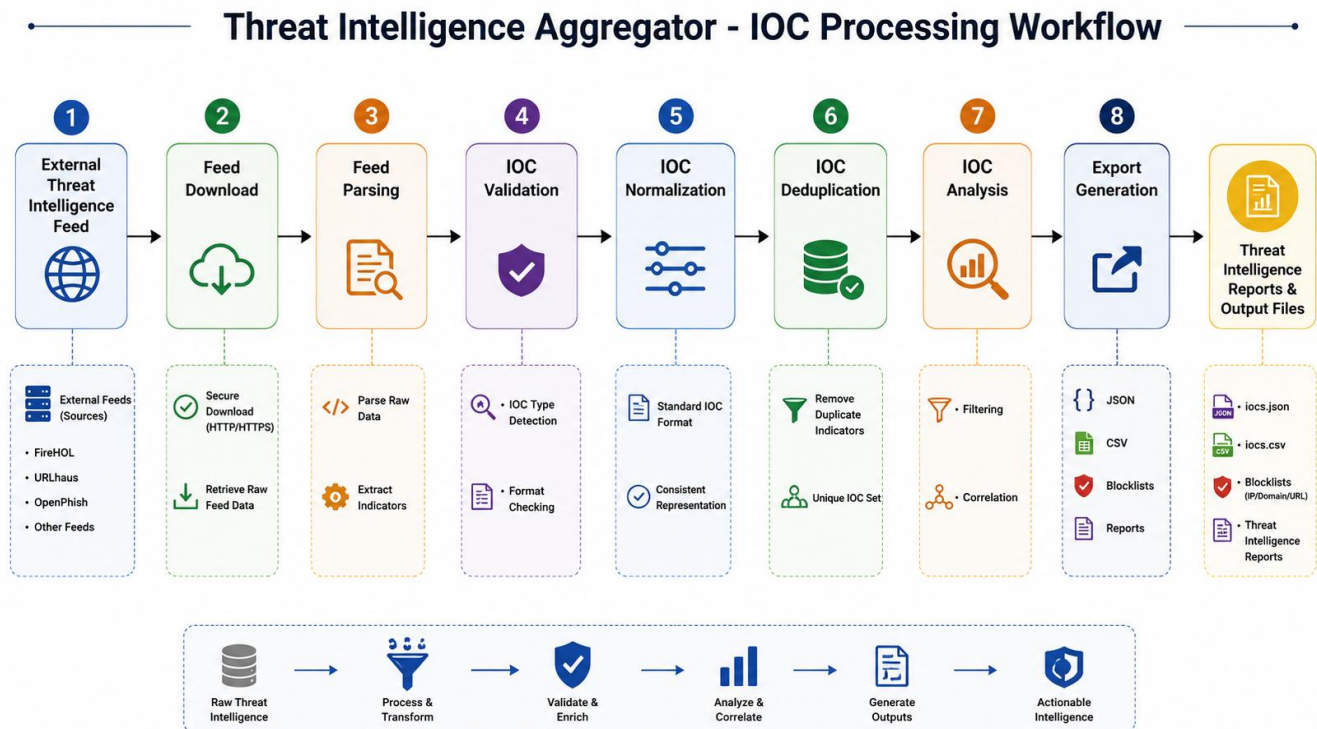


Figure 2. Threat Intelligence Processing Workflow

The application processes threat intelligence through a structured workflow that ensures consistent handling of data from multiple external sources.

The workflow begins by reading the YAML configuration file containing the list of threat intelligence feeds. The downloader retrieves the configured feeds from their respective sources. Depending on the feed format, the Parser Manager selects the appropriate parser (TXT, CSV, or JSON) to extract Indicators of Compromise.

The extracted indicators are then passed to the IOC Validator, which removes malformed or unsupported entries. Valid indicators are normalized into a common internal format to ensure consistency across different feeds. Duplicate indicators are identified and removed before the remaining data is correlated to determine which indicators appear across multiple sources.

Finally, the processed intelligence is exported into multiple output formats, including JSON files, CSV files, correlation reports, threat intelligence reports, and IOC-specific blocklists. Throughout execution, the logging system records operational information, warnings, and errors for monitoring and troubleshooting.

The workflow can be summarized as follows:

Configuration → Feed Download → Parsing → Validation → Normalization → Deduplication
→ Correlation → Report Generation → Export

9. Project Structure

The project is organized into multiple modules, with each directory responsible for a specific aspect of the application. This modular organization improves readability, maintainability, and future scalability.

A simplified project structure is shown below:

Threat_Intelligence_Aggregator/

```
|
|
|— configs/
|— data/
|— docs/
|— exports/
|— feeds/
|— logs/
|— reports/
|— src/
| |— analytics/
| |— cli/
| |— correlation/
| |— downloader/
| |— exporters/
| |— feeds/
| |— history/
| |— models/
| |— normalizer/
```

```

|   ├── parsers/
|   ├── reports/
|   ├── scheduler/
|   ├── utils/
|   └── validators/
|
|── tests/
|── main.py
|── requirements.txt
└── README.md

```

Description of Major Directories

Directory	Purpose
Configs	Stores YAML configuration files for threat feeds.
data	Contains downloaded or intermediate processing data.
docs	Project documentation and architecture/workflow diagrams.
exports	Stores generated JSON, CSV, and blocklist outputs.
feeds	Feed configuration and management resources.
logs	Application log files generated during execution.
reports	Generated threat intelligence and correlation reports.
src	Contains the complete application source code organized into modular packages.
tests	Automated unit tests for verifying application functionality

The modular directory structure separates configuration, processing logic, generated outputs, documentation, and testing, making the project easier to navigate and maintain.

10. Module Description

The Threat Intelligence Aggregator is composed of multiple independent modules, each responsible for a specific stage of the threat intelligence processing pipeline. This modular design follows the Single Responsibility Principle and improves maintainability, extensibility, and testing.

Configuration Module

The Configuration Module reads the YAML configuration file containing the list of threat intelligence feeds and their associated metadata. This allows new feeds to be added or modified without changing the application source code.

Downloader Module

The Downloader Module is responsible for retrieving threat intelligence feeds from external sources using HTTP/HTTPS. It manages download operations, handles network-related exceptions, and stores the retrieved content for further processing.

Feed Management Module

The Feed Management Module manages the configured threat intelligence sources and coordinates feed loading during execution. It ensures that each configured feed is processed according to its specified settings.

Scheduler Module

The Scheduler Module controls the execution sequence of feed processing tasks. It coordinates the overall workflow and ensures that each stage of the pipeline executes in the correct order.

Parser Manager

The Parser Manager determines the format of each downloaded feed and invokes the appropriate parser. This abstraction allows support for multiple feed formats without modifying other parts of the application.

TXT Parser

Processes plain text threat intelligence feeds by extracting supported Indicators of Compromise from text-based files.

CSV Parser

Reads CSV-formatted threat intelligence feeds and extracts IOC data from structured tabular records.

JSON Parser

Processes JSON-based threat intelligence feeds by reading structured objects and extracting supported indicators.

IOC Validator

The IOC Validator verifies that each extracted indicator conforms to the expected format. Invalid or malformed entries are discarded before further processing, improving the quality of the final dataset.

IOC Normalizer

Different threat intelligence feeds often represent the same information in different formats. The Normalizer converts valid indicators into a standardized internal representation, ensuring consistency across all processed data.

IOC Deduplication Module

The Deduplication Module identifies duplicate indicators collected from multiple feeds and removes redundant entries, reducing data size while improving report quality.

Correlation Module

The Correlation Module identifies indicators that appear in multiple threat intelligence feeds. This information is included in the correlation report to provide additional analytical insight into commonly reported threats.

Exporter Module

The Exporter Module generates multiple output formats, including:

- * JSON export
- * CSV export
- * Threat Intelligence Report
- * Correlation Report
- * IPv4 Blocklist
- * IPv6 Blocklist
- * IPv4 CIDR Blocklist
- * Domain Blocklist
- * URL Blocklist
- * MD5 Blocklist
- * SHA1 Blocklist
- * SHA256 Blocklist

Reporting Module

The Reporting Module summarizes processed threat intelligence, including feed statistics, IOC counts, and correlation information. The generated reports provide a concise overview of the processing results.

Command-Line Interface (CLI)

The CLI provides a simple interface for executing the application. Users can specify configuration files and initiate the threat intelligence processing workflow using command-line arguments.

Logging Module

The Logging Module records important execution events, processing statistics, warnings, and errors. These logs assist in monitoring application behavior and troubleshooting operational issues.

11. Implementation

The **Threat Intelligence Aggregator** was implemented using Python with a strong emphasis on modularity, maintainability, and software quality. Each functional component was developed independently and integrated into a unified processing pipeline.

The implementation begins by reading the feed configuration from a YAML file. Each configured feed is downloaded through the Downloader Module, after which the Parser Manager selects the appropriate parser based on the feed format. Supported formats include **TXT**, **CSV**, and **JSON**.

After parsing, the extracted Indicators of Compromise are validated to ensure they conform to supported IOC formats. Invalid entries are discarded before the remaining indicators are normalized into a common internal representation. This standardization simplifies downstream processing regardless of the original feed format.

Following normalization, duplicate indicators collected from multiple feeds are identified and removed. The remaining unique indicators are passed to the Correlation Module, which analyzes their occurrence across different sources. This correlation information is then incorporated into the generated reports.

Finally, the Exporter Module produces multiple output files, including structured **JSON** and **CSV datasets**, **IOC-specific blocklists**, **correlation reports**, and **comprehensive threat intelligence reports**. Throughout execution, the Logging Module records detailed operational information to assist with monitoring and debugging.

The implementation follows several software engineering best practices, including:

- * Clean Architecture
- * SOLID Principles
- * Modular Design
- * Dependency Injection
- * Type Hints
- * Comprehensive Logging
- * Automated Unit Testing
- * Clear Separation of Responsibilities

These practices improve code readability, simplify future maintenance, and provide a scalable foundation for additional functionality.

12. Testing & Validation

Testing was performed throughout the development process to verify the correctness, stability, and reliability of each application module. Automated testing was implemented using the **pytest** framework, allowing individual components to be tested independently before integration.

The test suite covers the core functionality of the application, including parser behavior, IOC validation, normalization, deduplication, correlation logic, exporters, and other critical processing components. External dependencies were isolated using mocking techniques where appropriate to ensure reliable and repeatable unit tests.

During the final validation phase, the complete test suite was executed using the following command:

```
python -m pytest
```

The final test results were:

Total Tests Executed: 76

Tests Passed: 76

Tests Failed: 0

In addition to automated testing, the complete application was validated through an end-to-end execution using the production configuration:

```
python main.py --run --config configs/feeds.yaml
```

The execution completed successfully with the following results:

Total Feeds Processed: 3

Successful Feeds: 3

Total Indicators of Compromise (IOCs): 26,289

Execution Time: Approximately 9 seconds

The successful execution confirmed that the complete processing pipeline—including downloading, parsing, validation, normalization, deduplication, correlation, reporting, and exporting—operated correctly and produced the expected outputs. These results demonstrate the reliability and readiness of Version 1.0.0 of the Threat Intelligence Aggregator.

13. Results

The Threat Intelligence Aggregator was successfully developed and validated as a modular threat intelligence processing toolkit. The final implementation met the defined project objectives by automating the collection, processing, validation, correlation, and export of threat intelligence obtained from multiple external sources.

During final system verification, the application successfully processed all configured threat intelligence feeds without errors. The runtime evaluation demonstrated that the toolkit is capable of efficiently handling a large volume of Indicators of Compromise while maintaining a lightweight execution profile.

The application successfully generated the following outputs:

- * JSON Export
- * CSV Export
- * Threat Intelligence Report
- * Correlation Report
- * IPv4 Blocklist
- * IPv6 Blocklist
- * IPv4 CIDR Blocklist
- * Domain Blocklist
- * URL Blocklist
- * MD5 Blocklist
- * SHA1 Blocklist
- * SHA256 Blocklist

These results demonstrate that the application is capable of processing multiple threat intelligence sources efficiently while producing structured outputs suitable for defensive cybersecurity operations. The successful completion of all automated tests further confirms the correctness and reliability of the implemented functionality.

14. Screenshots

Screenshot 1: Project Folder Structure

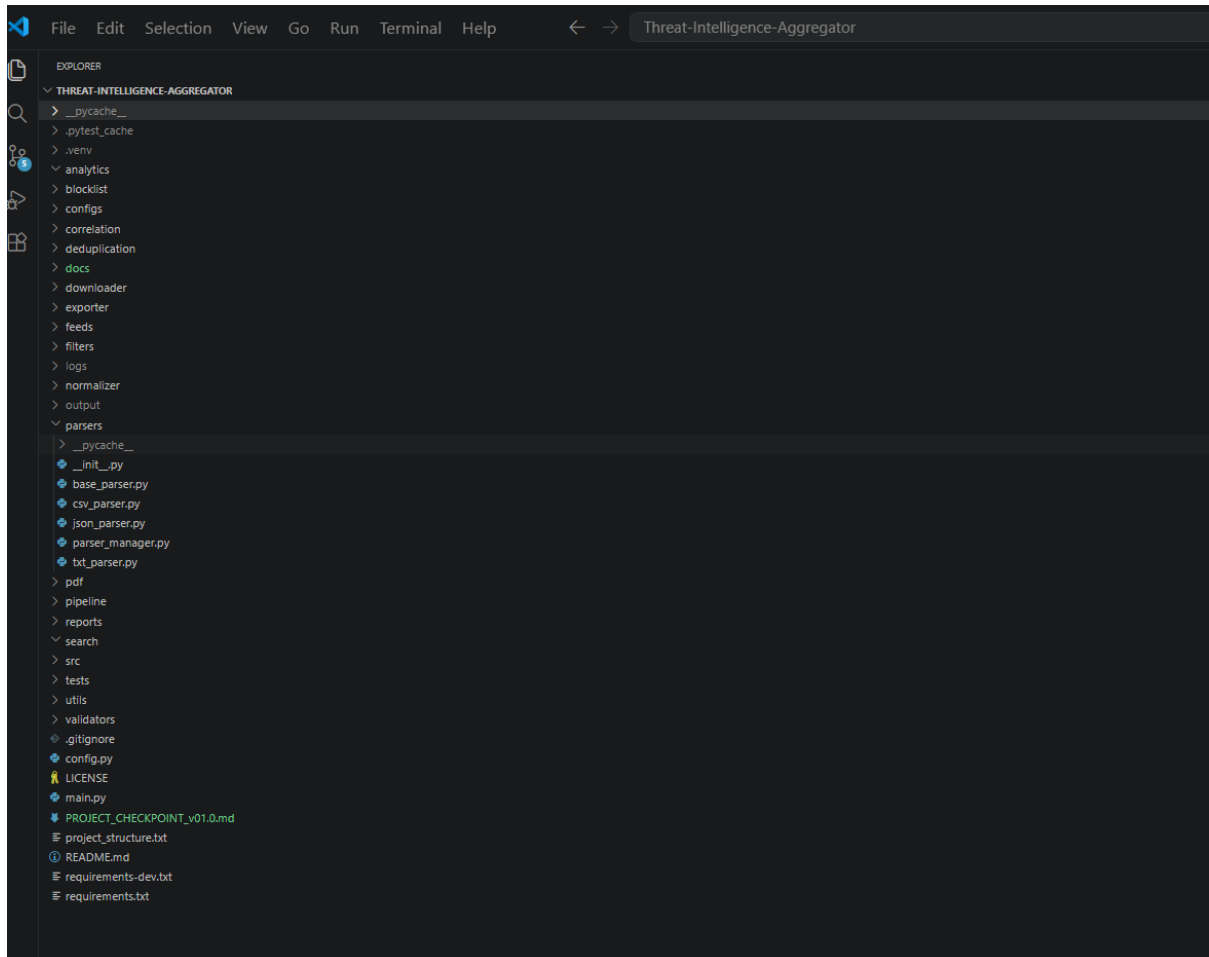


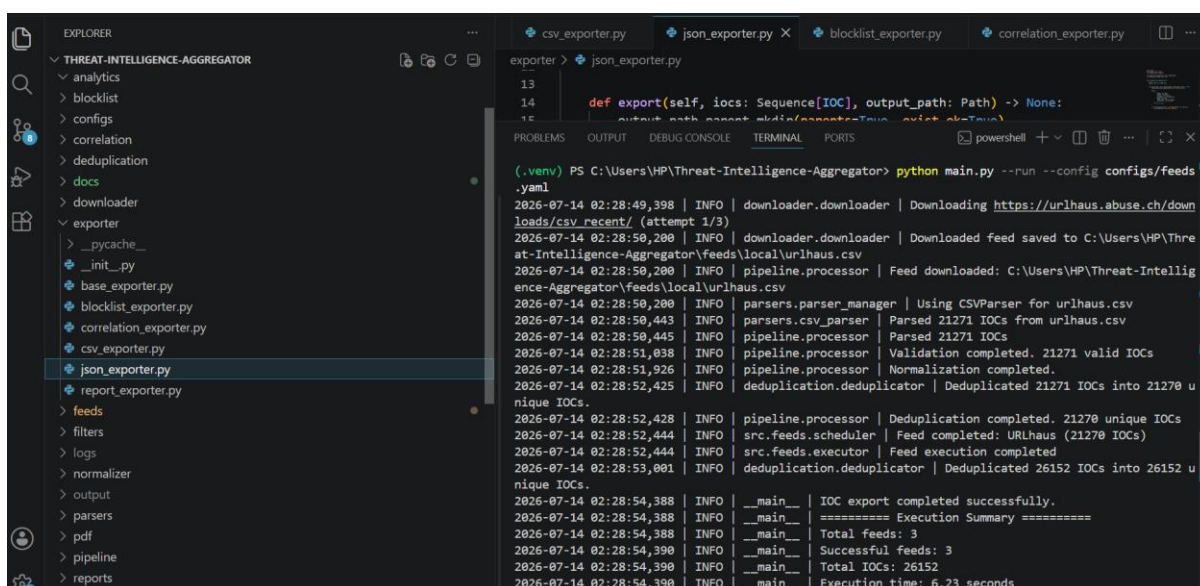
Figure 3. Project Directory Structure

Screenshot 2: Feed Configuration File

```
2026-07-14 02:22:56,317 | INFO | __main__ | IOC export completed successfully.
2026-07-14 02:22:56,318 | INFO | __main__ | ===== Execution Summary =====
2026-07-14 02:22:56,318 | INFO | __main__ | Total feeds: 3
2026-07-14 02:22:56,318 | INFO | __main__ | Successful feeds: 3
2026-07-14 02:22:56,319 | INFO | __main__ | Total IOCs: 26150
2026-07-14 02:22:56,319 | INFO | __main__ | Execution time: 5.88 seconds
(.venv) PS C:\Users\HP\Threat-Intelligence-Aggregator> |
```

Figure 4. Feed Configuration

Screenshot 3: Successful Application Execution



```
exporter > json_exporter.py
13
14
15
def export(self, iocs: Sequence[IOC], output_path: Path) -> None:
    output_path.parent.mkdir(exist_ok=True)

(.venv) PS C:\Users\HP\Threat-Intelligence-Aggregator> python main.py --run --config configs/feeds
.yaml
2026-07-14 02:28:49,398 | INFO | downloader.downloader | Downloading https://urlhaus.abuse.ch/down
loads/csv_recent/ (attempt 1/3)
2026-07-14 02:28:50,200 | INFO | downloader.downloader | Downloaded feed saved to C:\Users\HP\Thre
at-Intelligence-Aggregator\feeds\local\urlhaus.csv
2026-07-14 02:28:50,200 | INFO | pipeline.processor | Feed downloaded: C:\Users\HP\Threat-Intellig
ence-Aggregator\feeds\local\urlhaus.csv
2026-07-14 02:28:50,200 | INFO | parsers.parser_manager | Using CSVParser for urlhaus.csv
2026-07-14 02:28:50,443 | INFO | parsers.csv_parser | Parsed 21271 IOCs from urlhaus.csv
2026-07-14 02:28:50,445 | INFO | pipeline.processor | Parsed 21271 IOCs
2026-07-14 02:28:51,038 | INFO | pipeline.processor | Validation completed. 21271 valid IOCs
2026-07-14 02:28:51,926 | INFO | pipeline.processor | Normalization completed.
2026-07-14 02:28:52,425 | INFO | deduplication.deduplicator | Deduplicated 21271 IOCs into 21270 u
nique IOCs.
2026-07-14 02:28:52,428 | INFO | pipeline.processor | Deduplication completed. 21270 unique IOCs
2026-07-14 02:28:52,444 | INFO | src.feeds.scheduler | Feed completed: URLhaus (21270 IOCs)
2026-07-14 02:28:52,444 | INFO | src.feeds.executor | Feed execution completed
2026-07-14 02:28:53,001 | INFO | deduplication.deduplicator | Deduplicated 26152 IOCs into 26152 u
nique IOCs.
2026-07-14 02:28:54,388 | INFO | __main__ | IOC export completed successfully.
2026-07-14 02:28:54,388 | INFO | __main__ | ===== Execution Summary =====
2026-07-14 02:28:54,388 | INFO | __main__ | Total feeds: 3
2026-07-14 02:28:54,390 | INFO | __main__ | Successful feeds: 3
2026-07-14 02:28:54,390 | INFO | __main__ | Total IOCs: 26152
2026-07-14 02:28:54,390 | INFO | __main__ | Execution time: 6.23 seconds
```

Figure 5. Successful Threat Intelligence Processing

Screenshot 4: Automated Testing

```
test session starts
platform win32 -- Python 3.12.3, pytest-9.1.1, pluggy-1.6.0
rootdir: C:\Users\HP\Threat-Intelligence-Aggregator
collected 76 items

tests\test_correlation_engine.py . [ 1%]
tests\test_correlation_models.py .. [ 3%]
tests\test_correlation_rules.py .... [ 9%]
tests\test_deduplicator.py ..... [ 19%]
tests\test_downloader.py ... [ 23%]
tests\test_exporters.py .. [ 26%]
tests\test_feed_executor.py . [ 27%]
tests\test_feed_loader.py .. [ 30%]
tests\test_feed_model.py ..... [ 38%]
tests\test_feed_registry.py ..... [ 44%]
tests\test_history_manager.py .... [ 50%]
tests\test_json_parser.py ..... [ 56%]
tests\test_normalizer.py .... [ 61%]
tests\test_parser_manager.py .... [ 67%]
tests\test_pipeline_integration.py ..... [ 77%]
tests\test_processor.py . [ 78%]
tests\test_report.py . [ 80%]
tests\test_report_generator.py .. [ 82%]
tests\test_scheduler.py .... [ 88%]
tests\test_schema.py . [ 89%]
tests\test_statistics.py ..... [ 97%]
tests\test_summary.py . [ 98%]
tests\test_txt_parser.py . [100%]

===== 76 passed in 1.35s =====
(.venv) PS C:\Users\HP\Threat-Intelligence-Aggregator>
```

Caption: Figure 6. Automated Test Results

Screenshot 5: Generated Reports

```
▼ reports
  > __pycache__
  📄 __init__.py
  📄 report_generator.py
  📄 statistics.py
  📄 summary.py
```

Figure 7. Generated Reports

Screenshot 6: Exported Files

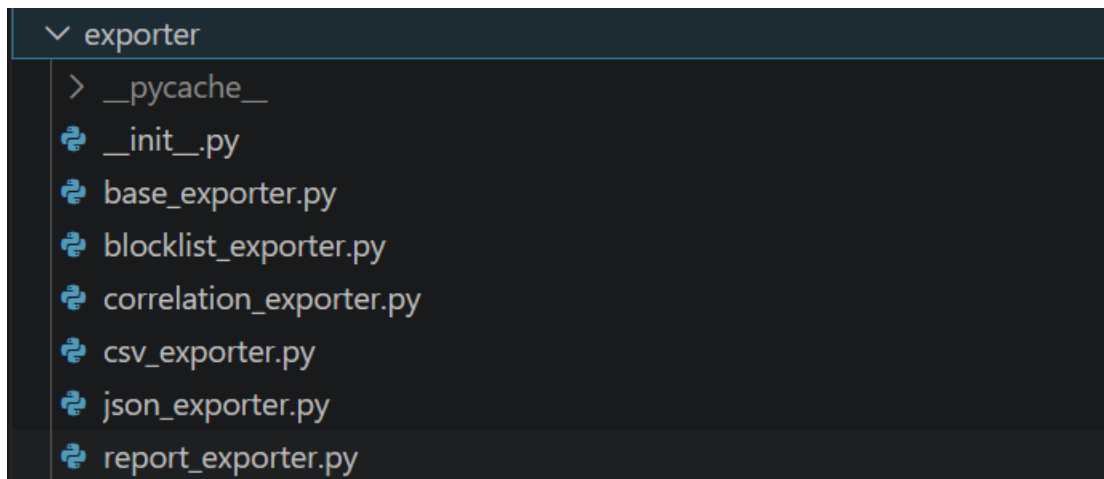


Figure 8. Generated Export Files

Including these screenshots provides visual evidence of the successful implementation, testing, and output generation of the Threat Intelligence Aggregator.

15. Challenges Faced

Several technical challenges were encountered during the development of the **Threat Intelligence Aggregator**. Addressing these challenges helped improve both the quality of the implementation and the overall software design.

One of the primary challenges was handling threat intelligence feeds published in different formats. Each feed followed its own structure, requiring separate parser implementations while maintaining a consistent processing workflow.

Another challenge involved validating multiple IOC types accurately. Different indicator categories such as IP addresses, domains, URLs, and cryptographic hashes required dedicated validation logic to ensure that only valid indicators entered the processing pipeline.

Duplicate indicators appearing across multiple feeds presented another difficulty. Efficient deduplication mechanisms were implemented to eliminate redundant entries while preserving useful correlation information.

Designing a modular architecture was also an important consideration. The application was structured using Clean Architecture principles and SOLID design principles to ensure that each module remained independent, testable, and easy to maintain.

Comprehensive testing required isolating individual components from external dependencies such as network requests. Automated unit tests and mocking techniques were used to verify module behavior under controlled conditions.

Finally, maintaining code readability while implementing multiple interconnected modules required careful organization, consistent coding practices, detailed logging, and clear separation of responsibilities throughout the project.

16. Future Scope

The current version of the Threat Intelligence Aggregator provides a stable and modular foundation for automated threat intelligence processing. Although the implemented features satisfy the objectives of Version 1.0.0, the architecture has been designed to support future enhancements with minimal modification to the existing codebase.

Several improvements can further increase the capabilities and practical applicability of the system.

Potential future enhancements include:

- i Integration with STIX/TAXII servers for standardized threat intelligence exchange.
- ii Support for additional IOC types, including email addresses.
- iii IOC severity and confidence scoring to assist in threat prioritization.
- iv Database integration for persistent storage and historical analysis of collected indicators.
- v REST API development for remote access and third-party integration.
- vi Interactive web dashboard for monitoring, visualization, and report management.
- vii Docker containerization to simplify deployment across different environments.
- viii Continuous Integration and Continuous Deployment (CI/CD) pipelines for automated testing and deployment.
- ix Support for additional threat intelligence feed formats.
- x Scheduled automatic execution using operating system schedulers or task automation services.
- xi Advanced analytics and statistical reporting for identifying long-term threat trends.

The modular architecture adopted in this project enables these enhancements to be incorporated without requiring significant changes to the existing implementation. This flexibility makes the Threat Intelligence Aggregator suitable for continued development into a more comprehensive threat intelligence platform.

17. Conclusion

The **Threat Intelligence Aggregator** was successfully designed, implemented, tested, and validated as a modular Python-based threat intelligence processing toolkit. The project achieved its primary objective of automating the collection, validation, normalization, correlation, and export of Indicators of Compromise obtained from multiple publicly available threat intelligence feeds.

The application demonstrates how a structured processing pipeline can transform heterogeneous threat intelligence data into consistent, organized, and reusable outputs. By supporting multiple feed formats and various IOC types, the system reduces manual effort while improving the accuracy and reliability of processed threat intelligence.

The project was developed using Clean Architecture, SOLID principles, dependency injection, modular design, comprehensive logging, and automated testing. These software engineering practices improve maintainability, scalability, and code quality while providing a strong foundation for future enhancements.

System validation confirmed the reliability of the implementation. The application successfully processed all configured threat intelligence feeds, generated the expected reports and blocklists, completed execution in approximately nine seconds, and passed all **76 automated unit tests without failures**.

Overall, the **Threat Intelligence Aggregator** provides an effective solution for automated threat intelligence processing and demonstrates the practical application of modern software engineering principles in cybersecurity. The modular architecture and extensible design ensure that the project can continue evolving to support additional capabilities as threat intelligence requirements grow.

18. References

The following references were consulted during the design, implementation, and testing of the Threat Intelligence Aggregator.

1. Python Software Foundation. ***Python 3 Documentation***. <https://docs.python.org/3/>
2. Pytest Development Team. ***pytest Documentation***. <https://docs.pytest.org/>
3. Requests Development Team. ***Requests: HTTP for Python***. <https://requests.readthedocs.io/>
4. PyYAML Documentation. ***PyYAML Documentation***. <https://pyyaml.org/>
5. MITRE Corporation. ***MITRE ATT&CK Framework***. <https://attack.mitre.org/>
6. MITRE Corporation. ***Common Vulnerabilities and Exposures (CVE)***. <https://cve.mitre.org/>
7. Abuse.ch. ***Threat Intelligence Feeds***. <https://abuse.ch/>
8. AlienVault OTX. ***Open Threat Exchange***. <https://otx.alienvault.com/>
9. Open Worldwide Application Security Project (OWASP). ***OWASP Foundation***. <https://owasp.org/>
10. National Institute of Standards and Technology (NIST). ***Cybersecurity Framework***. <https://www.nist.gov/cyberframework>

-----END OF REPORT-----